Spark Structured API for Data Transformation

This train will dive into the Structured API in Spark and how to apply it to ingest and transform data efficiently.

Learning Objective

By the end of this train, you should be able to:

- Understand and apply the Spark Structured API to ingest and transform data.

Spark Structured API and RDD

As previously mentioned, most of the processing in Spark is done using the Structured API (pyspark.sql). However, underneath that, and in the first versions of Spark, the main version and method to interact with Spark was through the RDD API, which is still supported and allows some functionality that is not possible with the higher levels of abstraction.

Spark RDD

The RDD is the lowest level of abstraction in Spark. It has three characteristics: dependencies, partitions, and the compute function. All three of these are integral to RDD programming on which all high-level APIs are constructed:

- A list of dependencies instructs Spark on how an RDD is constructed with its required inputs. This enables resiliency, by allowing Spark to recreate the RDD from scratch if required.
- Partitions allow Spark to split the processing between executors.
- RDD has a compute function that produces an iterator for the data that will be stored in the RDD. The iterator will iterate over the data to perform the computations it is instructed to do.

While this schema is simple and elegant, there are a couple of issues, including that most of the computation is opaque to Spark, meaning the operation cannot be inspected and optimized. Spark does not know the data type, and can thus not optimally serialize the object.

First, we have to create an RDD. RDDs are defined by running the `parallelize()` method on the defined SparkContext, and unlike DataFrames have to be viewed using the `collect()` method.

```
rdd = sc.parallelize(
    [('pink', 10), ('blue', 9), ('green', 8), ('gold', 7), ('black', 6),
('pink', 7), ('blue', 6)])
rdd.collect()
```

Output:

```
Out[9]: [('pink', 10),
         ('blue', 9),
         ('green', 8),
         ('gold', 7),
         ('black', 6),
         ('pink', 7),
         ('blue', 6)]
```

We can inspect various parts of the RDD, such as getting the number of partitions the data reside in:

```
rdd.getNumPartitions()
```

Output:

```
Out[10]: 8
```

We can perform various transformations on the RDD object, albeit a bit more difficult than working with Pandas DataFrames.

Here we map a lambda function to each value and access the tuples in the RDD as you would rows:

```
rdd.map(lambda y: y[1] * 2).collect()
```

Output:

```
Out[11]: [20, 18, 16, 14, 12, 14, 12]
```

If you want to preserve the RDD structure, you have to specify the tuple structure again:

```
rdd.map(lambda y: (y[0], y[1] * 2)).collect()
```

Output:

```
Out[12]: [('pink', 20),
          ('blue', 18),
          ('green', 16),
          ('gold', 14),
          ('black', 12),
          ('pink', 14),
          ('blue', 12)]
```

You can apply additional functions, such as sorting the returned RDD:

```
rdd.map(lambda y: (y[0], y[1] * 2)).sortByKey().collect()
```

Output:

```
Out[13]: [('black', 12),
          ('blue', 18),
          ('blue', 12),
          ('gold', 14),
          ('green', 16),
          ('pink', 20),
          ('pink', 14)]
```

Some of the other operations on the RDD are a lot less intuitive. `reduceByKey()` is a method that merges the values for each key with the function specified. In this case, the lambda function indicates what should happen to the values of the keys that are matched:

```
rdd.reduceByKey(lambda a,b: a+b).collect()
```

Output:

```
Out[14]: [('pink', 17), ('blue', 15), ('black', 6), ('green', 8), ('gold', 7)]
```

The RDD API is not a very intuitive interface to work with and does not have all the optimisations that are contained within the higher-level abstractions. It does allow full customisation, which means that it can be tuned completely, and in that sense does provide optimisation.

If you are more interested in the RDD API, spend some time reading the documentation..

Conclusion

This lesson introduced the Spark Structured API for efficient data ingestion and transformation. While the RDD API provides low-level control, the Structured API offers higher-level abstractions, optimizations, and better performance for large-scale data processing. Understanding the strengths and trade-offs of both APIs helps you choose the right tool for your data processing needs in Spark.

▼Key Takeaways

- **Takeaway 1:** Spark Structured API offers high-level abstractions for efficient data processing.
- **Takeaway 2:** RDD API provides low-level control but requires more manual optimization.
- **Takeaway 3:** Spark DataFrames are optimized for better performance than RDDs.
- **Takeaway 4:** Transformations in RDDs, like `map`, `reduceByKey`, and `sortByKey`, allow complex data manipulations..
- **Takeaway 5:** Choosing between RDD and Structured API depends on the specific processing requirements and optimization needs.